
Historias de Usuario MVP – TradeUp TCG

Proyecto: Parcial #1 – MVP E-Commerce de Cartas TCG

Estudiantes: Samuel González, Isabella Linares

Curso: Desarrollo y Gestión de Software – UTP

Fecha: Junio 2026

SECCIÓN 1: Funcionalidades YA Implementadas

(Basado en el código real del repositorio)

HU-01 – Sincronización de Usuario con Clerk

Como usuario nuevo, quiero que mi cuenta de Clerk se sincronice automáticamente con la base de datos del sistema, para tener un perfil en la plataforma con rol y reputación.

- El endpoint `POST /api/auth/sync` crea o actualiza el documento de usuario en MongoDB.
- Se extrae el email primario verificado desde Clerk.
- El rol por defecto al crear es `buyer`; roles superiores no se sobrescriben en re-sincronización.
- Se retorna: `id`, `clerkId`, `email`, `username`, `role`, `stripeConnectStatus`, `reputation`.

HU-02 – Autenticación y Control de Acceso por Roles

Como sistema, quiero proteger los endpoints con guards de Clerk (`requireAuth`, `requireAdmin`, `requireSeller`), para que solo usuarios autorizados accedan a cada funcionalidad.

- `requireAuth` valida el JWT de Clerk en todos los endpoints privados.
- `requireAdmin` bloquea con 403 a usuarios sin rol admin.
- `requireSeller` bloquea con 403 a usuarios que no son vendedores.
- Los usuarios baneados (`isBanned: true`) reciben 403 en endpoints de compra/oferta.

HU-03 – Búsqueda de Cartas en el Catálogo

Como usuario, quiero buscar cartas del catálogo TCG por texto y juego, para encontrar las cartas que me interesan.

- `GET /api/catalog/search?q=&game=` realiza búsqueda full-text con `$text` de MongoDB.
- Se puede filtrar por juego: `pokemon`, `yugioh`, `onepiece`, `dragonball`, `mtg`, `other`.
- Los resultados se ordenan por relevancia (`textScore`) y están paginados.
- La búsqueda mínima es de 2 caracteres; devuelve 400 si es menor.

HU-04 – Ver Detalle de una Carta del Catálogo

Como usuario, quiero ver los datos completos de una carta específica, para conocer su rareza, set, número y juego.

- `GET /api/catalog/:id` devuelve la carta por su ID de MongoDB.
- Incluye: name, set, setCode, cardNumber, rarity, game, imageUrl, language.
- Devuelve 404 si la carta no existe.
- El endpoint es público, no requiere autenticación.

HU-05 – Administrar Catálogo de Cartas (Admin)

Como administrador, quiero crear, editar y eliminar cartas del catálogo oficial, para mantener la base de cartas disponibles actualizada.

- `POST /api/catalog` crea una carta con validación Zod; evita duplicados por game+setCode+cardNumber.
- `PATCH /api/catalog/:id` actualiza campos con `runValidators: true`.
- `DELETE /api/catalog/:id` elimina permanentemente la carta.
- Solo usuarios con `requireAdmin` pueden acceder a estos endpoints.

HU-06 – Solicitudes de Cartas al Catálogo (Usuarios)

Como usuario autenticado, quiero solicitar la adición de una carta que no está en el catálogo, para que el admin la revise y apruebe.

- `POST /api/catalog/requests` crea una solicitud asociada al usuario.
- Se evitan solicitudes duplicadas pendientes del mismo usuario para la misma carta.
- El admin puede aprobar (`PATCH /requests/:id/approve`) o rechazar (`PATCH /requests/:id/reject`).
- Al aprobar, la carta se agrega automáticamente al catálogo.

HU-07 – Publicar Listing de Carta para Vender/Intercambiar

Como vendedor, quiero publicar una carta que deseo vender o intercambiar, indicando condición, fotos, precio y cartas que acepto a cambio, para que otros usuarios puedan hacerme ofertas.

- `POST /api/listings` requiere `requireSeller` ; necesita catalogCardId y condition.
- El listing puede tener askingPrice (venta) y/o wantsCards (intercambio).
- Se almacenan hasta N fotos por listing.
- El listing se crea en estado `active` .

HU-08 – Explorar Listings del Marketplace

Como usuario, quiero ver todos los listings activos con filtros de juego, condición y búsqueda, para encontrar cartas disponibles en el marketplace.

- `GET /api/listings` filtra por estado `active` , game, condition y texto.

- Cada listing muestra la topBid actual (mayor oferta de dinero pendiente) calculada con agregación.
- Los resultados incluyen datos del vendedor (username, reputation, reviewCount).
- Los resultados están paginados.

HU-09 – Hacer Oferta sobre un Listing

Como comprador, quiero hacer una oferta de dinero, cartas o mixta sobre un listing, para negociar la adquisición de una carta.

- `POST /api/offers` valida con Zod: tipo (money/cards/mixed), monto y cartas ofrecidas.
- No se puede hacer oferta sobre el propio listing.
- Solo se permite una oferta pendiente por usuario por listing.
- Las ofertas tienen fecha de expiración calculada con `getOfferExpiryDate()`.

HU-10 – Aceptar o Rechazar Oferta (Vendedor)

Como vendedor, quiero aceptar o rechazar ofertas recibidas sobre mis listings, para concretar o desestimar transacciones.

- `POST /api/offers/:id/accept` valida que el usuario sea el seller; verifica que la oferta no esté expirada.
- Al aceptar, todas las demás ofertas pendientes del listing se marcan como `declined`.
- El listing cambia a estado `sold` (money) o `traded` (cards).
- Se crea automáticamente un `Transaction` con status `pending`.

HU-11 – Cancelar Oferta (Comprador)

Como comprador, quiero cancelar una oferta pendiente que hice, para retirar mi propuesta si cambié de opinión.

- `POST /api/offers/:id/cancel` valida que el usuario sea el buyer de la oferta.
- Solo se pueden cancelar ofertas en estado `pending`.
- Devuelve 403 si otro usuario intenta cancelar.
- La oferta pasa a estado `cancelled`.

HU-12 – Tienda Oficial B2C (Admin vende cartas directamente)

Como administrador, quiero publicar cartas en la tienda oficial con precio fijo, stock, condición y estado de gradeo, para venderlas directamente a compradores.

- `POST /api/store` (admin) acepta multipart con hasta 5 fotos; valida tipos MIME y tamaño máximo 15MB.
- Soporte para cartas gradadas (isGraded, gradeValue, gradeCompany) y selladas (isSealed).
- `GET /api/store` soporta filtros: game, condition, minPrice, maxPrice, graded, q, sort.
- `DELETE /api/store/:id` desactiva el ítem (soft delete con `isActive: false`).

HU-13 – Comprar Carta de la Tienda Oficial

Como comprador, quiero comprar una carta de la tienda oficial con pago por Stripe, para adquirirla a precio fijo.

- `POST /api/store/:id/buy` crea un `PaymentIntent` de Stripe con `capture` automático.
- Se crea un `Transaction` de tipo `b2c` con `snapshot` del ítem (nombre, imagen, set, condición).
- El endpoint valida stock disponible y estado activo del ítem.
- Se retorna el `clientSecret` para que el frontend procese el pago.

HU-14 – Pago C2C entre Usuarios con Stripe Connect

Como comprador, quiero pagar una carta a otro usuario a través de Stripe Connect, para que la plataforma distribuya el pago automáticamente al vendedor.

- `POST /api/payments/c2c-intent` crea un `PaymentIntent` con `capture_method: manual`.
- Si el vendedor tiene Stripe Connect activo, se configura `transfer_data` y `application_fee_amount`.
- Se calcula comisión de plataforma con `calculateCommission(amount)`.
- Se valida que el monto mínimo sea 100 centavos.

HU-15 – Webhooks de Stripe para Actualizar Transacciones

Como sistema, quiero procesar los webhooks de Stripe para actualizar automáticamente el estado de las transacciones, para no depender del cliente al confirmar pagos.

- `webhooks.ts` procesa eventos de Stripe firmados.
- Existe un route dedicado `/api/webhooks` para manejar eventos de Stripe.
- Los eventos actualizan el estado de `Transaction` en MongoDB.
- La firma del webhook de Stripe se valida antes de procesar.

HU-16 – Chat entre Comprador y Vendedor

Como usuario, quiero chatear con la contraparte de una transacción aceptada, para coordinar la entrega o el intercambio de cartas.

- `chat.ts` gestiona los mensajes entre participantes de una transacción.
- El chat está asociado a un `transactionId`.
- Solo los participantes de la transacción acceden al chat.
- Los mensajes se almacenan en MongoDB.

HU-17 – Reseñas entre Usuarios

Como comprador o vendedor, quiero dejar una reseña a la contraparte después de una transacción, para construir la reputación del sistema.

- `reviews.ts` gestiona la creación y consulta de reseñas.

- Solo transacciones con `reviewEligible: true` permiten reseña.
- Las reseñas afectan el campo `reputation` y `reviewCount` del usuario.
- No se puede reseñar dos veces la misma transacción.

HU-18 – Notificaciones de Usuario

Como usuario, quiero recibir notificaciones del sistema sobre ofertas, transacciones y eventos importantes, para estar al tanto de la actividad en mi cuenta.

- `notifications.ts` expone endpoints para consultar notificaciones.
- Las notificaciones están asociadas al usuario autenticado.
- Se pueden consultar notificaciones pendientes/no leídas.
- El endpoint requiere autenticación (`requireAuth`).

HU-19 – Historial de Transacciones

Como usuario, quiero ver el historial completo de mis transacciones (compras, ventas, trades), para llevar control de mi actividad en la plataforma.

- `transactions.ts` expone endpoints de consulta por usuario.
- Cada transacción tiene: tipo (b2c, c2c_money, c2c_trade, c2c_mixed), estado, montos, shippingStatus.
- Se puede ver el detalle de una transacción por ID.
- El historial está paginado y ordenado por fecha.

HU-20 – Panel de Administración (Backoffice)

Como administrador, quiero acceder a un panel web de administración, para gestionar usuarios, catálogo, tienda y solicitudes desde una interfaz visual.

- Existe la app `apps/backoffice` como parte del monorepo.
- El backoffice consume los endpoints de admin de la API.
- Se gestiona desde aquí: usuarios, catálogo, solicitudes de cartas, tienda.
- El acceso requiere rol admin verificado con Clerk.

HU-21 – Gestión de Usuarios (Admin)

Como administrador, quiero listar, ver el detalle, banear y modificar usuarios, para mantener el orden en la plataforma.

- `admin.ts` y `users.ts` exponen endpoints de gestión de usuarios.
 - Se puede banear un usuario (`isBanned: true`) lo que le impide hacer ofertas o compras.
 - Se visualiza: username, email, role, reputation, reviewCount, stripeConnectStatus.
 - Solo admins acceden a estos endpoints.
-

SECCIÓN 2: Funcionalidades Faltantes – Production-Ready

HU-22 – Refresh Token y Gestión de Sesión Segura

Como usuario autenticado, quiero que mi sesión se gestione de forma segura con tokens de larga duración, para no perder el acceso inesperadamente.

- Clerk maneja el refresh token; se debe configurar correctamente la duración de sesión.
- El logout revoca la sesión en Clerk y limpia cookies/localStorage.
- Los tokens expirados retornan 401 con mensaje claro.
- Se documenta el flujo de sesión en el README.

HU-23 – Rate Limiting en Endpoints Sensibles

Como operador, quiero limitar peticiones por IP en endpoints de auth y pagos, para prevenir abuso y ataques de fuerza bruta.

- Los endpoints de `/api/auth/sync` y `/api/payments/*` tienen límite de 20 req/min por IP.
- Las peticiones excedentes retornan HTTP 429 con header `Retry-After`.
- El rate limiting se configura mediante variables de entorno.
- Los IPs bloqueados se registran en logs.

HU-24 – Roles Granulares y Permisos Configurables [MCP]

Como administrador, quiero gestionar roles (admin, seller, buyer, moderator) y sus permisos desde la plataforma, para controlar el acceso sin necesidad de redesplicue.

Tool MCP: `manage_roles`

Autenticación: JWT Clerk con metadata `role: admin`

Permite al agente: Listar usuarios por rol, promover/degradar un usuario, consultar permisos activos.

- Existe interfaz en backoffice para cambiar el rol de un usuario.
- El cambio de rol se sincroniza con Clerk metadata.
- El cambio queda registrado en audit log con quién lo hizo y cuándo.
- Los sellers deben pasar por un flujo de verificación antes de activarse.

HU-25 – Onboarding de Stripe Connect para Vendedores [MCP]

Como vendedor, quiero conectar mi cuenta de Stripe para recibir pagos directamente, para activar mi capacidad de venta C2C en la plataforma.

Tool MCP: `get_seller_stripe_status`

Autenticación: JWT Clerk scope usuario autenticado

Permite al agente: Consultar si un vendedor tiene Stripe Connect activo, iniciar el flujo de onboarding, verificar estado.

- Se genera un AccountLink de Stripe Connect para el flujo de onboarding.

- Al completar, `stripeConnectStatus` se actualiza a `active`.
- Los vendedores sin Stripe Connect activo pueden vender pero el pago queda retenido en plataforma.
- El estado de Stripe Connect es visible en el perfil del vendedor.

HU-26 – Captura Manual de Pagos C2C tras Confirmación

Como sistema, quiero capturar el pago C2C manualmente solo cuando ambas partes confirman la recepción, para proteger al comprador y al vendedor.

- El PaymentIntent C2C usa `capture_method: manual`; la captura se ejecuta con `/api/payments/c2c-capture`.
- La captura se permite solo cuando la transacción tiene estado `completed`.
- Si la transacción se cancela, el PaymentIntent se cancela y no se cobra.
- Existe un tiempo límite para capturar antes de que Stripe expire el intent.

HU-27 – Reembolsos y Disputas [MCP]

Como administrador, quiero procesar reembolsos y gestionar disputas entre usuarios, para resolver conflictos y mantener la confianza en la plataforma.

Tool MCP: `refund_transaction`

Autenticación: JWT con rol admin

Permite al agente: Consultar transacciones disputadas, emitir un refund en Stripe, actualizar estado de transacción.

- `POST /api/admin/transactions/:id/refund` procesa el reembolso vía Stripe.
- Solo admins pueden emitir reembolsos.
- El reembolso repone el stock del ítem en tienda B2C.
- El usuario recibe notificación del reembolso procesado.

HU-28 – Logs Estructurados de Aplicación

Como operador DevOps, quiero que la API emita logs estructurados en JSON con nivel, requestId y contexto, para analizar incidentes en producción.

- Se implementa Pino o Winston para logging estructurado.
- Cada request genera un `requestId` único propagado en todos los logs del flujo.
- Los errores 5xx registran el stack trace completo.
- Los logs se pueden exportar a servicios externos (Datadog, Logtail).

HU-29 – Endpoint de Salud y Métricas [MCP]

Como operador, quiero un endpoint `/health` que reporte el estado de MongoDB, Stripe y servicios externos, para monitorear la plataforma en tiempo real.

Tool MCP: `get_system_health`

Autenticación: API Key de servicio

Permite al agente: Consultar estado de DB, latencia promedio, tasa de errores, servicios degradados.

- `GET /health` retorna status de MongoDB, Clerk y Stripe.
- Se expone `/metrics` compatible con Prometheus.
- El endpoint responde en < 200ms incluso bajo carga.
- Un dashboard en Grafana visualiza uptime y latencia.

HU-30 – Alertas Automáticas de Errores [MCP]

Como operador, quiero recibir alertas cuando la tasa de errores supere un umbral o un servicio externo falle, para reaccionar rápido ante incidentes.

Tool MCP: `get_active_alerts`

Autenticación: API Key con scope `ops:alerts`

Permite al agente: Listar alertas activas, obtener detalles del incidente, silenciar una alerta temporalmente.

- Las alertas se envían por Slack o correo al superar el umbral configurado.
- Se detecta automáticamente si Stripe o Clerk no responden.
- Las alertas incluyen: severidad, componente, tiempo de inicio.
- Se resuelven automáticamente cuando el servicio se recupera.

HU-31 – Manejo Global de Excepciones

Como desarrollador, quiero que todos los errores no manejados retornen respuestas estructuradas consistentes, para facilitar el debugging y la experiencia del cliente.

- Existe un middleware global de error en Hono que captura todas las excepciones.
- Las respuestas de error incluyen: statusCode, message, timestamp, path.
- Los errores 500 en producción no exponen stack traces al cliente.
- Los errores de validación Zod retornan detalle de qué campos fallaron.

HU-32 – Pruebas Unitarias de Lógica de Negocio

Como desarrollador, quiero pruebas unitarias con cobertura mínima del 70% en la lógica crítica (offers, payments, transactions), para asegurar el correcto funcionamiento.

- Los archivos de prueba `.test.ts` cubren: cálculo de comisiones, validaciones de oferta, cambios de estado.
- Las pruebas mockean MongoDB y Stripe.
- La cobertura mínima es 70% en statements.
- Las pruebas se ejecutan con `bun test` o `npm test`.

HU-33 – Pruebas E2E de Flujos Críticos

Como QA, quiero pruebas E2E que validen los flujos completos de listing → oferta → aceptación → pago, para garantizar que el sistema funciona de extremo a extremo.

- Las pruebas E2E usan una base de datos MongoDB de test separada.
- Se prueban: sync de usuario, crear listing, hacer oferta, aceptar oferta, pago Stripe (modo test).
- Las pruebas se ejecutan en CI antes de cada despliegue.

- Stripe en modo test con webhooks simulados.

HU-34 – Pipeline CI/CD con GitHub Actions

Como equipo, quiero que cada push a main ejecute lint, pruebas y build automáticamente, para detectar errores antes de llegar a producción.

- El workflow ejecuta: typecheck → lint → test → build de todas las apps del monorepo.
- El pipeline falla si alguna prueba falla o hay errores de TypeScript.
- El despliegue automático a staging solo ocurre si el pipeline es verde.
- Se notifica al equipo si el pipeline falla en main.

HU-35 – Despliegue con Docker y Variables de Entorno Seguras

Como DevOps, quiero desplegar la plataforma con Docker Compose y gestionar secretos de forma segura, para garantizar portabilidad entre ambientes.

- Existe `docker-compose.yml` para producción con servicios: API, MongoDB, Redis.
- La imagen Docker usa multi-stage build para minimizar tamaño final.
- Las variables sensibles (Stripe keys, Clerk keys, MongoDB URI) se inyectan como secretos de Docker o Vault.
- El `.env.example` (ya existente en el repo) documenta todas las variables requeridas.

HU-36 – Backup Automático de MongoDB

Como operador, quiero que MongoDB se respalde automáticamente cada noche, para poder recuperar datos ante un fallo catastrófico.

- Se ejecuta un `mongodump` automático cada 24 horas vía cron job o Atlas Backup.
- Los backups se almacenan encriptados en S3 o equivalente.
- Se puede restaurar en menos de 1 hora desde el backup más reciente.
- Los backups de más de 30 días se eliminan automáticamente.

HU-37 – Sistema de Expiración Automática de Ofertas

Como sistema, quiero que las ofertas expiradas se marquen automáticamente, para que el marketplace no tenga ofertas "fantasma" inactivas.

- Un job periódico (cron o BullMQ) revisa ofertas con `expiresAt < now()` y status `pending`.
- Las ofertas expiradas se marcan como `expired` en lote.
- El vendedor recibe notificación cuando sus ofertas expiran.
- El job se ejecuta cada hora y registra cuántas ofertas procesó.

HU-38 – Caché de Catálogo y Listings con Redis [MCP]

Como usuario, quiero que el catálogo y los listings públicos respondan rápido incluso con muchos usuarios simultáneos, para una experiencia fluida.

Tool MCP: `invalidate_cache`

Autenticación: API Key con scope `ops:cache`

Permite al agente: Forzar invalidación del caché de catálogo o de un listing específico, consultar hit rate, ver TTLs activos.

- El catálogo se cachea en Redis con TTL de 5 minutos.
- Los listings públicos se cachean con TTL de 1 minuto.
- Al crear/actualizar/eliminar una carta o listing, el caché se invalida automáticamente.
- El tiempo de respuesta cacheado es < 50ms.

HU-39 – Búsqueda Avanzada de Listings [MCP]

Como usuario, quiero buscar listings con filtros combinados de juego, rareza, condición, precio y tipo de intercambio aceptado, para encontrar exactamente lo que busco.

Tool MCP: `search_listings`

Autenticación: Pública (sin auth) o JWT para resultados personalizados

Permite al agente: Buscar listings disponibles con múltiples filtros, ordenar por precio/fecha/topBid, paginar resultados.

- La búsqueda combina filtros de CatalogCard (game, rarity) y Listing (condition, price, wantsCards).
- Los resultados incluyen topBid actual de cada listing.
- La búsqueda es insensible a mayúsculas y acentos.
- Los resultados responden en < 200ms con índices apropiados.

HU-40 – Notificaciones por Email Transaccional

Como usuario, quiero recibir correos de confirmación para eventos importantes (oferta recibida, oferta aceptada, pago confirmado), para estar informado aunque no esté conectado.

- Se integra SendGrid o Resend para email transaccional.
- Los correos tienen plantillas HTML con branding de TradeUp TCG.
- Se envían para: nueva oferta recibida, oferta aceptada, pago confirmado, transacción completada.
- Los correos fallidos se reintentan 3 veces con BullMQ.

HU-41 – Notificaciones en Tiempo Real vía WebSockets [MCP]

Como usuario, quiero recibir notificaciones en tiempo real cuando alguien me hace una oferta o acepta la mía, para reaccionar inmediatamente sin recargar la página.

Tool MCP: `get_user_notifications`

Autenticación: JWT Clerk con scope usuario autenticado

Permite al agente: Listar notificaciones no leídas de un usuario, marcarlas como leídas, obtener conteo de pendientes.

- Se implementa WebSocket o Server-Sent Events en la API Hono.
- Las notificaciones se emiten en tiempo real al usuario conectado.
- Las notificaciones no leídas persisten en MongoDB y se muestran al reconectar.
- El usuario puede marcar notificaciones como leídas desde el frontend.

HU-42 – Internacionalización (i18n) de Mensajes de la API

Como usuario hispanohablante, quiero que los mensajes de error y validación aparezcan en español, para entenderlos mejor.

- Se implementa i18n con soporte para `es` y `en`.
- El idioma se selecciona con el header `Accept-Language`.
- Los mensajes de validación Zod se traducen al idioma del cliente.
- El idioma por defecto es `en`; `es` es el secundario soportado.

HU-43 – Documentación de API con OpenAPI/Swagger [MCP]

Como desarrollador externo o agente IA, quiero acceder a la especificación completa de la API de TradeUp TCG, para integrar o automatizar flujos de la plataforma.

Tool MCP: `get_api_schema`

Autenticación: Pública para lectura; JWT para ejecutar requests

Permite al agente: Obtener lista de todos los endpoints, schemas de request/response, ejemplos, autenticación requerida por endpoint.

- Se genera un schema OpenAPI 3.0 compatible con Swagger UI.
- El schema se expone en `GET /api/docs` (Swagger UI) y `GET /api/openapi.json`.
- Todos los endpoints tienen descripción, parámetros y ejemplos de respuesta documentados.
- El schema se exporta como JSON/YAML para importar en Claude Code o Codex.

HU-44 – Audit Log de Acciones Administrativas [MCP]

Como administrador, quiero que todas las acciones sensibles (bans, cambios de rol, reembolsos, modificaciones de catálogo) queden en un log de auditoría inmutable, para tener trazabilidad completa.

Tool MCP: `get_audit_log`

Autenticación: JWT con rol admin

Permite al agente: Buscar acciones por usuario, tipo o rango de fechas; exportar logs en CSV; detectar patrones sospechosos de abuso.

- Se registra: quién (adminId), qué acción, sobre qué entidad (userId, listingId, etc.), cuándo y desde qué IP.
- El audit log solo permite inserts, nunca updates ni deletes.
- Se pueden filtrar logs por tipo de acción y rango de fechas.
- Los logs son exportables en JSON o CSV.

HU-45 – Reputación y Sistema de Confianza [MCP]

Como usuario, quiero que mi reputación se actualice automáticamente al recibir reseñas, y quiero ver el score de confianza de un vendedor antes de hacerle una oferta, para tomar decisiones informadas.

Tool MCP: `get_user_reputation`

Autenticación: Pública para consulta; JWT para ver historial detallado propio

Permite al agente: Consultar reputación de un usuario por ID, obtener sus reseñas recientes, calcular score de confianza.

- La reputación se recalcula automáticamente al crear una reseña (promedio ponderado).
- El perfil público muestra: reputación, reviewCount, transacciones completadas, tiempo en la plataforma.
- Los vendedores con reputación < 3.0 reciben badge de advertencia en sus listings.
- Se puede ver el detalle de reseñas recibidas por cualquier usuario.

HU-46 – Análisis de Mercado y Precios de Referencia [MCP]

Como usuario, quiero ver el precio promedio de mercado de una carta basado en transacciones recientes de la plataforma, para saber si una oferta es justa.

Tool MCP: `get_market_price`

Autenticación: Pública con rate limiting

Permite al agente: Obtener precio promedio, mínimo y máximo de una carta en los últimos 30 días, ver tendencia de precio, comparar con el precio de un listing.

- El precio de mercado se calcula sobre las últimas 20 transacciones completadas de esa carta.
- El historial de precios de los últimos 30 días es visible por carta.
- Los datos de mercado se actualizan al completarse cada transacción.
- La consulta es pública pero tiene rate limiting de 60 req/min por IP.

HU-47 – Panel de Analytics para Admin [MCP]

Como administrador, quiero ver métricas del negocio (volumen de transacciones, revenue de comisiones, cartas más populares, usuarios activos), para tomar decisiones basadas en datos.

Tool MCP: `get_platform_analytics`

Autenticación: JWT con rol admin

Permite al agente: Obtener resumen de transacciones por período, revenue de comisiones, top cartas por volumen, tasa de conversión de listings a ventas.

- El dashboard muestra: transacciones del día/semana/mes, revenue de comisiones, listings activos.
- Los datos se agregan con MongoDB aggregation pipelines.
- Las métricas se pueden filtrar por rango de fechas.
- El endpoint responde en < 500ms con índices apropiados.

HU-48 – Moderación de Listings y Reportes de Usuarios

Como usuario, quiero reportar un listing fraudulento o un usuario problemático, para que el equipo de moderación pueda actuar.

- `POST /api/listings/:id/report` y `POST /api/users/:id/report` crean reportes.
- Los reportes se almacenan con: reporterId, targetId, motivo, descripción.
- Los admins pueden ver reportes pendientes en el backoffice.
- Al acumular 3 reportes, el listing/usuario se marca para revisión prioritaria.

HU-49 – Paginación con Cursor y Optimización de Queries

Como desarrollador, quiero que todos los endpoints de listado usen cursor-based pagination e índices

optimizados, para mantener rendimiento con millones de documentos.

- Se implementa cursor-based pagination en `/api/listings` , `/api/catalog/search` y `/api/store` .
- Los campos de búsqueda frecuente tienen índices en MongoDB (game, status, seller, createdAt).
- El tiempo de respuesta de queries paginadas es < 100ms con índices.
- Se documenta el plan de índices en `BASE_DE_DATOS.md` .

HU-50 – Accesibilidad del Frontend Web

Como usuario con discapacidad visual, quiero que la interfaz web de TradeUp TCG cumpla con estándares de accesibilidad WCAG 2.1 AA, para poder usarla con tecnologías asistivas.

- Todos los componentes interactivos tienen atributos `aria-*` correctos.
- El contraste de colores cumple con ratio mínimo de 4.5:1.
- La navegación por teclado funciona en flujos críticos (buscar, hacer oferta, comprar).
- Se realiza auditoría con Lighthouse Accessibility con score ≥ 85 .

HU-51 – Gestión de Envíos y Tracking [MCP]

Como comprador, quiero recibir actualizaciones del estado de envío de mi compra en tienda B2C, para saber cuándo llegará mi carta.

Tool MCP: `get_shipment_status`

Autenticación: JWT del usuario comprador o admin

Permite al agente: Consultar el `shippingStatus` de una transacción, obtener número de tracking, ver historial de cambios de estado de envío.

- Las transacciones B2C tienen campo `shippingStatus` (pending, shipped, delivered).
- El admin puede actualizar el estado de envío y agregar número de tracking.
- El comprador recibe notificación cuando el estado de envío cambia.
- El historial de cambios de estado se almacena en la transacción.

HU-52 – Whitelist/Blacklist de IPs y Protección Anti-fraude

Como operador, quiero detectar y bloquear comportamientos fraudulentos automáticamente, para proteger la plataforma de abuso.

- Se detectan patrones: múltiples cuentas desde el mismo IP, offers masivas en poco tiempo, usuarios con IPs de VPN/Tor.
- Los IPs sospechosos se agregan a una lista de monitoreo.
- Los admins pueden banear IPs desde el backoffice.
- Las detecciones automáticas se registran en el audit log.

Tabla Resumen – Historias MCP

#	Título	Tool MCP	Autenticación
HU-24	Roles Granulares	<code>manage_roles</code>	JWT rol admin
HU-25	Stripe Connect Sellers	<code>get_seller_stripe_status</code>	JWT usuario
HU-27	Reembolsos y Disputas	<code>refund_transaction</code>	JWT admin
HU-29	Salud del Sistema	<code>get_system_health</code>	API Key servicio
HU-30	Alertas de Errores	<code>get_active_alerts</code>	API Key <code>ops:alerts</code>
HU-38	Caché Redis	<code>invalidate_cache</code>	API Key <code>ops:cache</code>
HU-39	Búsqueda de Listings	<code>search_listings</code>	Pública / JWT opcional
HU-41	Notificaciones RT	<code>get_user_notifications</code>	JWT usuario
HU-43	Documentación OpenAPI	<code>get_api_schema</code>	Pública
HU-44	Audit Log	<code>get_audit_log</code>	JWT admin
HU-45	Reputación de Usuario	<code>get_user_reputation</code>	Pública / JWT propio
HU-46	Precios de Mercado	<code>get_market_price</code>	Pública (rate limited)
HU-47	Analytics de Plataforma	<code>get_platform_analytics</code>	JWT admin
HU-51	Estado de Envío	<code>get_shipment_status</code>	JWT usuario / admin